Exam 1 is on Oct 13: Two Weeks from Today

- Exam covers material from the 6 assigned papers ***only***. These papers are posted on the USC Brightspace.
- Oct 13:
  - 5-6 pm: Review. Ask all your questions.
  - 6-7 pm: Exam 1 in the lecture hall.
- Prepare for the exam early: Read the papers, watch the lectures, ask questions. Make sure you understand the material.

- Two exams from the 2024 offering of this course are posted on the USC Brightspace for Week 8.
  - **Important:** The 2025 exam content has been updated. The papers **SDM** and **DynamoDB** on the 2024 exams are **replaced** by **ER (Entity-Relationship) Modeling** and **Grasper** in the current 2025 offering.

Hybrid-Range Partitioning Strategy

by Shahram Ghandeharizadeh and David DeWitt

CSCI 585

Hybrid-Range Partitioning Strategy

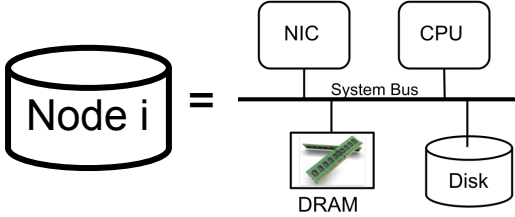
Key contributions:

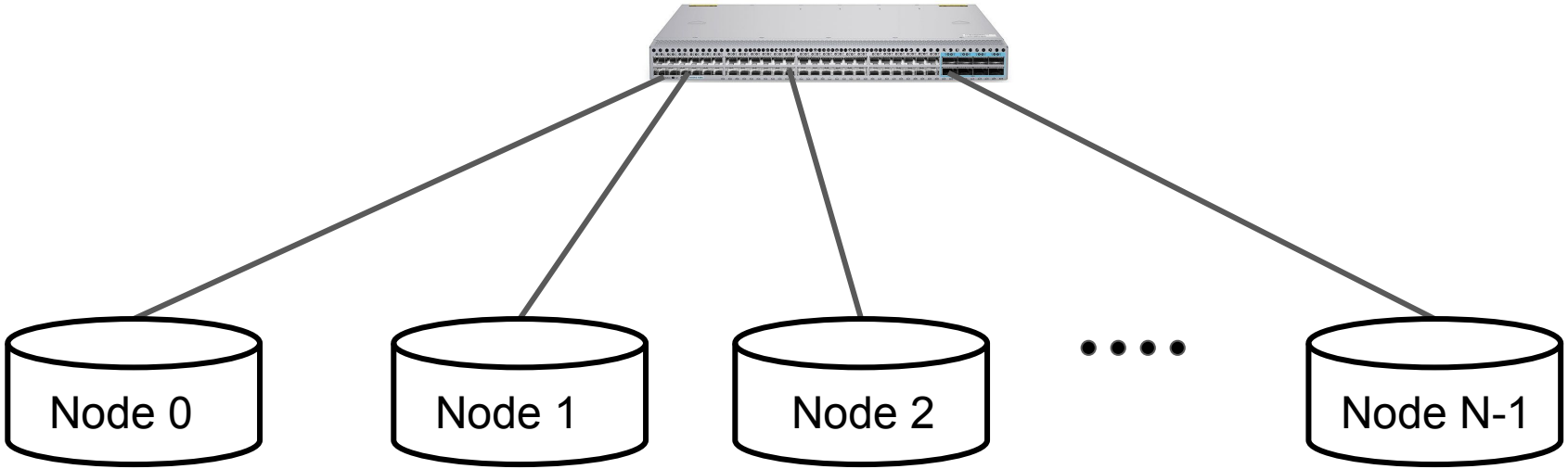
- Controlled placement of data based on the processing capability of the underlying hardware and the system workload. Why?
  - Parallelism has overhead. Prevent this overhead from dominating response time.
  - Maximize degree of parallelism to balance the workload across nodes evenly.

- Modern transactional storage managers such as LevelDB, FoundationDB and Nova-LSM use declustering techniques similar to the Hybrid-Range Partitioning Strategy.
  - Hybrid-range pre-dates these systems by 10 years.

Background: Scan, Index Structures, Good and Bad of Parallel Query Processing

A Shared-Nothing Architecture

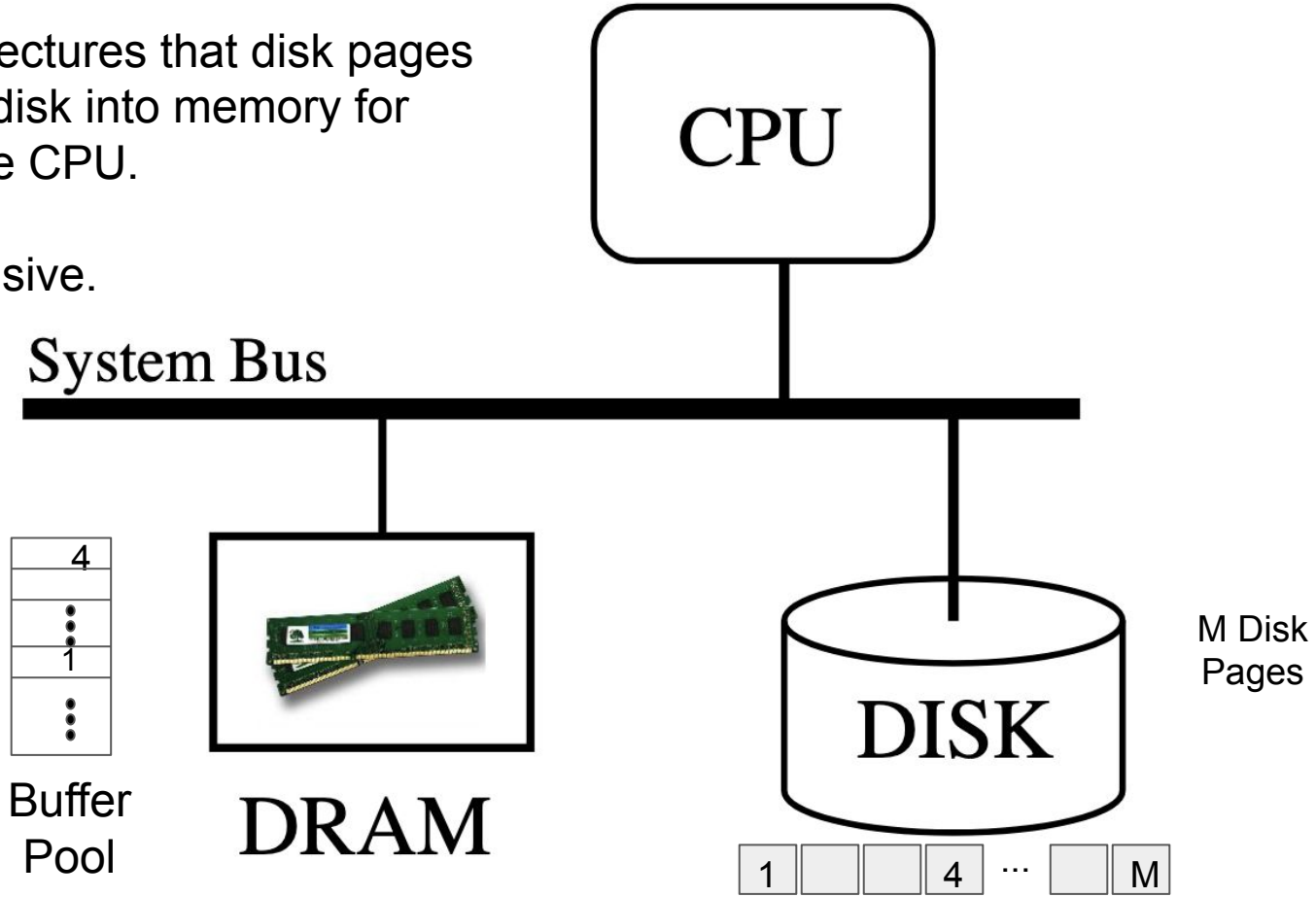




Disk Input/Output (I/O)

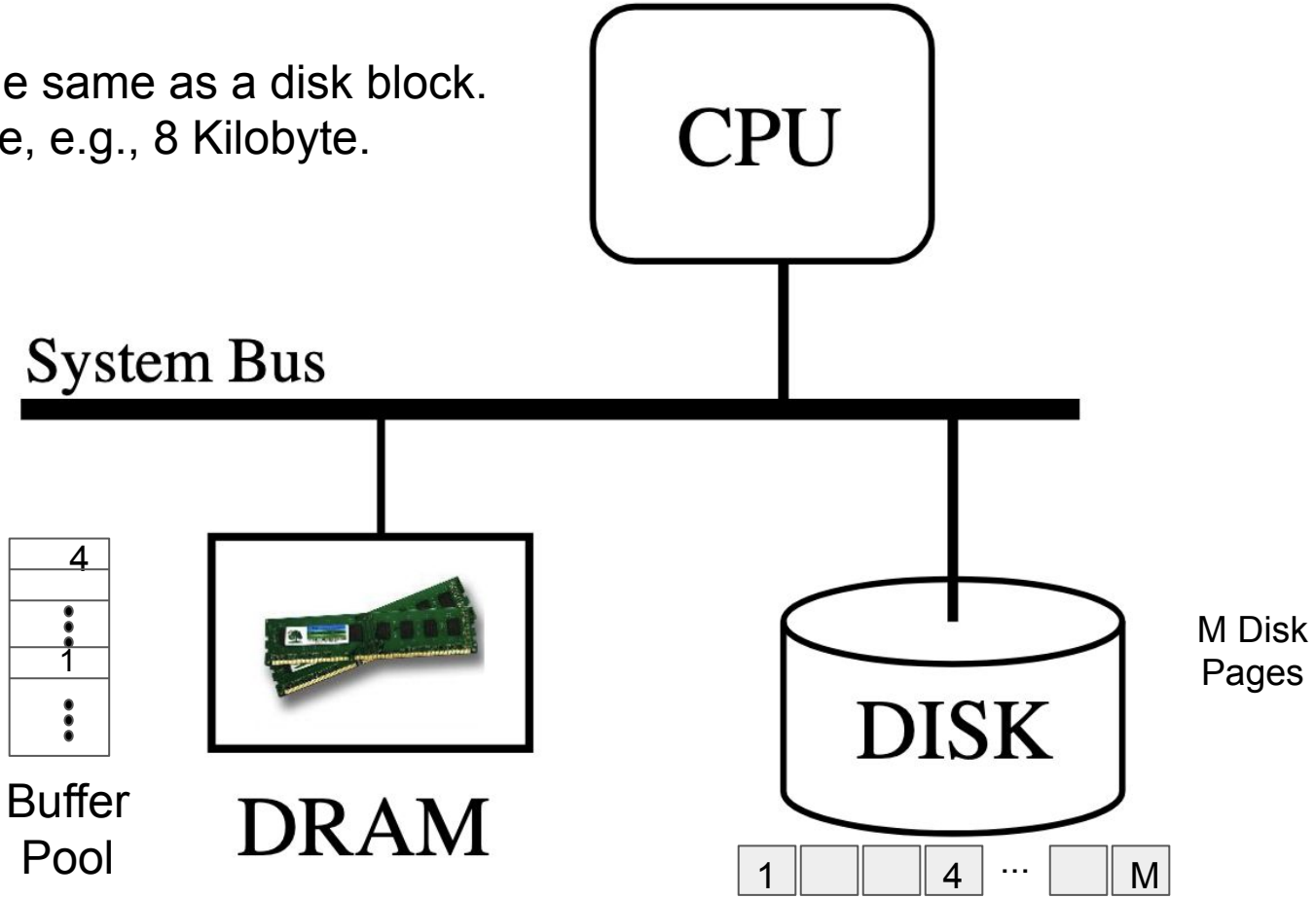
Recall from last lectures that disk pages are staged from disk into memory for processing by the CPU.

Disk I/O is expensive.



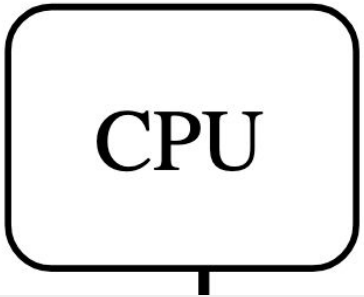
Disk Input/Output (I/O)

A disk page is the same as a disk block.
It has a fixed size, e.g., 8 Kilobyte.



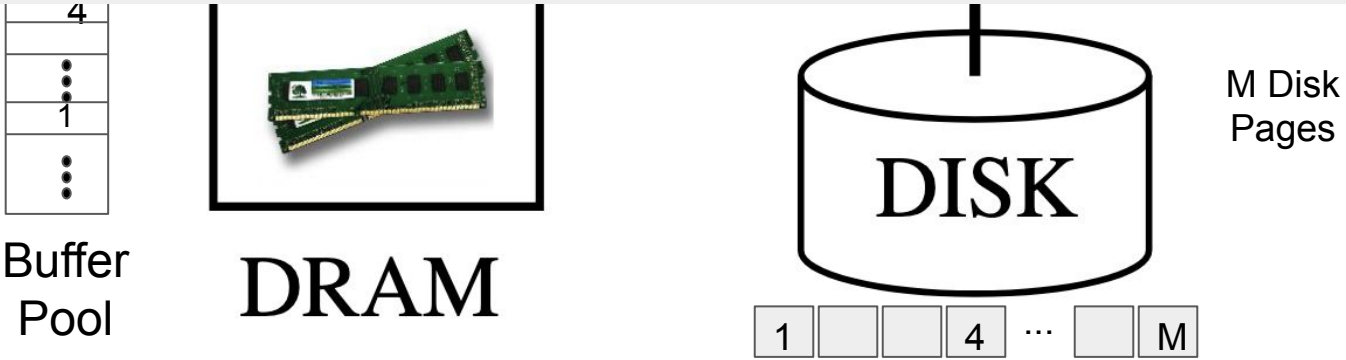
Disk Input/Output (I/O)

A disk page is the same as a disk block.
It has a fixed size, e.g., 8 Kilobyte.

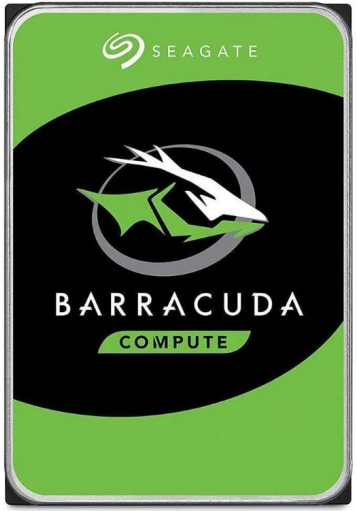


Once a disk page is stagged in the buffer pool, repeated references for it will find the page in the buffer pool. A buffer pool hit results in fast processing.

Disk block = Disk page = Page = Block (has a unique address on disk).



Disk Cache



Roll over image to zoom in

Seagate 1TB Desktop HDD Hard Drive - Internal (ST1000DM003)

Visit the Seagate Store

4.5 ★★★★½ 369 ratings | Search this page

200+ bought in past month

-7% $38.00

Typical price: $40.99

Save up to 5% with business pricing. Sign up for a free Amazon Business account

Digital Storage Capacity	1 TB
Hard Disk Interface	Serial ATA-600
Connectivity Technology	SATA
Brand	Seagate
Special Feature	Backward Compatible
Hard Disk Form Factor	3.5 Inches
Hard Disk Description	Mechanical Hard Disk
Compatible Devices	PC

64 MB disk cache reduces latency for sequential reads. How? When a disk page is read (and as long as there are no other pending read/write disk page requests), the disk controller anticipates that the next page will be referenced and reads it into the cache. This is referred to as **read-ahead** or **prefetching**.

TERMINOLOGY

- Once staged in memory, a disk block is a sequence of zero and ones, termed bits:

  0101111110011010101010110000000000…..00000

- A byte is eight contiguous bits:

  0101111**11001101**0101010110000000000…..00000

TERMINOLOGY

- Bits:

  01011111100110101010101100000000000…..00000

- A byte is eight contiguous bits:

  01011111100110**10101010**1100000000000…..00000

TERMINOLOGY

- Bits:

  01011111100110101010101100000000000…..00000

- A byte is eight contiguous bits:

  01011111100110101010101100000000000…..00000

TERMINOLOGY

- Bits:

  01011111100110101010101100000000000…..00000

- A byte is eight contiguous bits:

  01011111100110101010101100000000000…..00000

- 1024 bytes is one Kilobyte (KB), e.g., text in a college textbook is in the order of kilobytes.

TERMINOLOGY

- Bits:

  01011111100110101010101100000000000…..00000

- A byte is eight contiguous bits:

  01011111100110101010101100000000000…..00000

- 1024 bytes is one Kilobyte (KB), e.g., text of a college textbook.
- 1024 KB is one Megabyte (MB), e.g., a high resolution photograph is in the order of megabytes.

TERMINOLOGY

- Bits:

  01011111100110101010101100000000000…..00000

- A byte is eight contiguous bits:

  01011111100110101010101100000000000…..00000

- 1024 bytes is one Kilobyte (KB), e.g., text of a college textbook.
- 1024 KB is one Megabyte (MB), e.g., a high resolution photograph.
- 1024 MB is one Gigabyte (GB), e.g., a DVD quality movie is in the order of gigabytes.

Feature: 32.6 Gig
2:06:01
Disc: 50GB (dual-layered)

TERMINOLOGY

- Bits:

  01011111100110101010101100000000000…..00000

- A byte is eight contiguous bits:

  01011111100110101010101100000000000…..00000

- 1024 bytes is one Kilobyte (KB), e.g., text of a college textbook.
- 1024 KB is one Megabyte (MB), a high resolution photograph.
- 1024 MB is one Gigabyte (GB), e.g., a DVD quality movie.
- 1024 GB is one Terabyte (TB), e.g., all text in the library of congress.

TERMINOLOGY

- Bits:

  01011111100110101010101100000000000…..00000

- A byte is eight contiguous bits:

  01011111100110101010101100000000000…..00000

- 1024 bytes is one Kilobyte (KB), e.g., text of a college textbook.
- 1024 KB is one Megabyte (MB), a high resolution photograph.
- 1024 MB is one Gigabyte (GB), e.g., a DVD quality movie.
- 1024 GB is one Terabyte (TB), e.g., all text in the library of congress.
- 1024 TB is one Petabyte (PB), e.g., entire LoC multimedia collection ~ 7 PB.

TERMINOLOGY

- Bits:

  01011111100110101010101100000000000…..00000

- A byte is eight contiguous bits:

  01011111100110101010101100000000000…..00000

- 1024 bytes is one Kilobyte (KB), e.g., text of a college textbook.
- 1024 KB is one Megabyte (MB), a high resolution photograph.
- 1024 MB is one Gigabyte (GB), e.g., a DVD quality movie.
- 1024 GB is one Terabyte (TB), e.g., all text in the library of congress.
- 1024 TB is one Petabyte (PB), e.g., entire LoC multimedia collection ~ 7 PB.
- 1024 PB is one Exabyte (XB), e.g., record all phone conversations in a year.

TERMINOLOGY

- Bits:

  01011111100110101010101100000000000…..00000

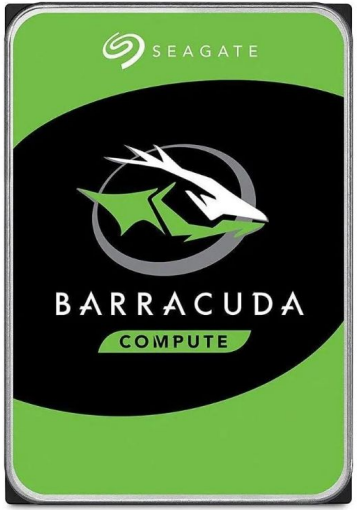
- A byte is eight contiguous bits:

  01011111100110101010101100000000000…..00000

- 1024 bytes is one Kilobyte (KB), e.g., text of a college textbook.
- 1024 KB is one Megabyte (MB), a high resolution photograph.
- 1024 MB is one Gigabyte (GB), e.g., a DVD quality movie.
- 1024 GB is one Terabyte (TB), e.g., all text in the library of congress.
- 1024 TB is one Petabyte (PB), e.g., entire LoC multimedia collection ~ 7 PB.
- 1024 PB is one Exabyte (XB), e.g., record all phone conversations in a year.
- 1024 XB is one Zetabyte (ZB), e.g., all uncompressed medical data.

1 TB Disk Drive Shows up as ~907 GB

2024



Roll over image to zoom in

Seagate 1TB Desktop HDD Hard Drive - Internal (ST1000DM003)

Visit the Seagate Store

4.5 ★★★★½ 369 ratings | Search this page

200+ bought in past month

-7% $38.00

Typical price: $40.99

Save up to 5% with business pricing. Sign up for a free Amazon Business account

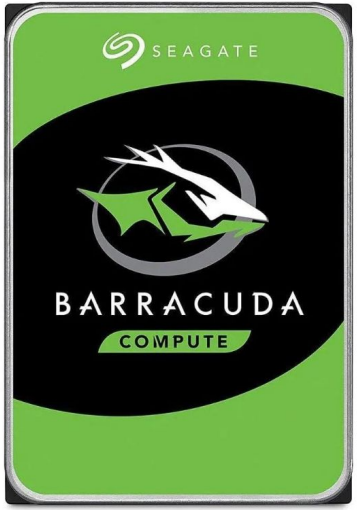
Digital Storage Capacity	1 TB
Hard Disk Interface	Serial ATA-600
Connectivity Technology	SATA
Brand	Seagate
Special Feature	Backward Compatible
Hard Disk Form Factor	3.5 Inches
Hard Disk Description	Mechanical Hard Disk
Compatible Devices	PC

64 MB Cache: Reduces latency for sequential reads.

2025: $39.73

Where did ~93 GB go?

1 TB Disk Drive Shows up as ~ 907 GB



Roll over image to zoom in

Seagate 1TB Desktop HDD Hard Drive - Internal (ST1000DM003)

Visit the Seagate Store

4.5 ★★★★½ 369 ratings | Search this page

200+ bought in past month

-7% $38.00

Typical price: $40.99

Save up to 5% with business pricing. Sign up for a free Amazon Business account

Digital Storage Capacity	1 TB
Hard Disk Interface	Serial ATA-600
Connectivity Technology	SATA
Brand	Seagate
Special Feature	Backward Compatible
Hard Disk Form Factor	3.5 Inches
Hard Disk Description	Mechanical Hard Disk
Compatible Devices	PC

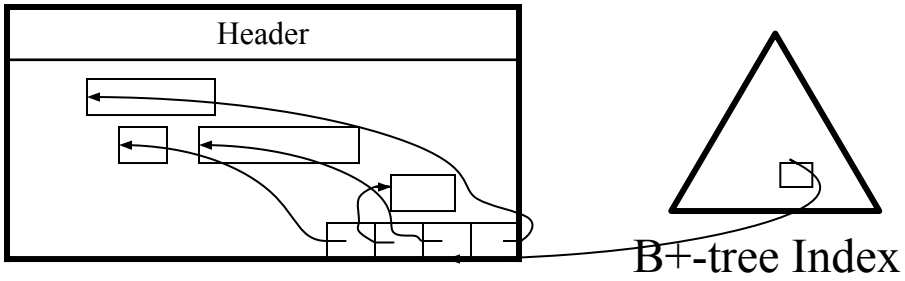
Operating Systems: 1 TB = 1024 bytes/KB * 1024 KB/MB * 1024 MB/GB * 1024 GB/TB

Disk Manufacturers: 1 TB = 1000 bytes/KB * 1000 KB/MB * 1000 MB/GB * 1000 GB/TB

OS TB - Disk TB = 92.67 GB

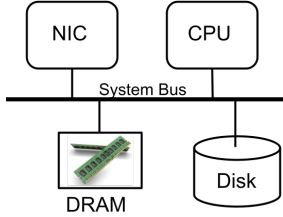
Physical Organization of Records in Blocks

- Indexed Heap:



- Each page consists of an array of pointers, each pointer points to a record within the block.
- A record is located by providing its block number and index in the pointer array. This combination is called a Tuple Identifier (TID) or a Record Identifier (RID).
- Insertion and deletion are easy, accomplished by manipulating the pointer array.
- The contents of a block may be reorganized without affecting external pointers pointing to records. RID does not change when records are moved around within a block.

Implementation of б: Heap Scan



- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Shideh	4	4000	1

- $б_{Salary=30,000}$(Employee)

SS#	Name	Age	Salary	dno
4	Kathy	30	30000	5

- Process the select operator using a file scan (linear scan)

  F1 = Open the file corresponding to Employee

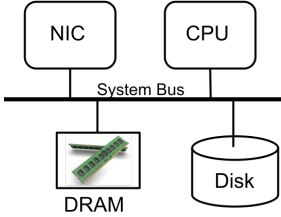
  P = read first page of F1

  While P is not null

  For each record in P, if the record satisfies the selection predicate then produce as output

  P = read next page of F1 /* P becomes null when EoF is reached */

Implementation of б: Heap Scan



- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Shideh	4	4000	1

- $б_{Salary=30,000}$(Employee)

SS#	Name	Age	Salary	dno
4	Kathy	30	30000	5

- Process the select operator using a file scan (linear scan)

  F1 = Open the file corresponding to Employee

  P = read first page of F1

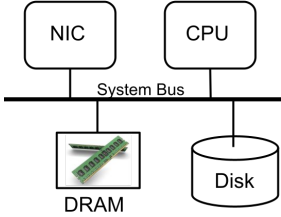
  While P is not null

    For each record in P, if the record satisfies the selection predicate then produce as output

    P = read next page of F1

Fetch the page from disk if not in the buffer pool

Implementation of б: Heap Scan



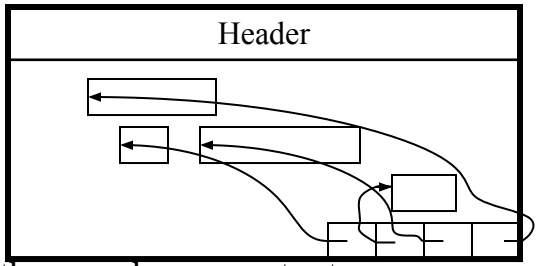
- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Shideh	4	4000	1

- $б_{Salary=30,000}$(Employee)

SS#	Name	Age	Salary	dno
4	Kathy	30	30000	5

- Process the select operator using a file scan (linear scan)



F1 = Open the file corresponding to Employee
P = read first page of F1
While P is not null
    For each record in P, if the record satisfies the selection predicate then produce as output
    P = read next page of F1

Index Structures

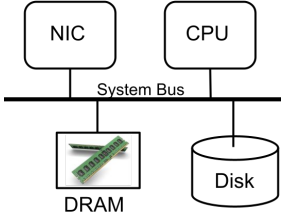
A hash index processes exact match predicates referencing the indexed attribute efficiently.

```
SELECT *
FROM Emp
WHERE SS# = 396-43-1909
```

A B+-Tree index structures processes range predicates referencing the indexed attribute efficiently.

```
SELECT *
FROM Emp
WHERE age > 20 and age < 21
```

HEAP FILE ORGANIZATION



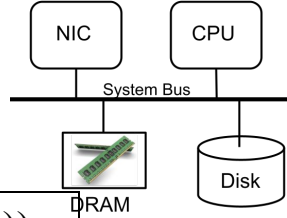
- Assume a student table: Student(name, age, gpa, major)
  t(Student) = 16
  P(Student) = 4

Bob, 21, 3.7, CS	Kane, 19, 3.8, ME	Louis, 32, 4, LS	Chris, 22, 3.9, CS
Mary, 24, 3, ECE	Lam, 22, 2.8, ME	Martha, 29, 3.8, CS	Chad, 28, 2.3, LS
Tom, 20, 3.2, EE	Chang, 18, 2.5, CS	James, 24, 3.1, ME	Leila, 20, 3.5, LS
Kathy, 18, 3.8, LS	Vera, 17, 3.9, EE	Pat, 19, 2.8, EE	Shideh, 16, 4, CS

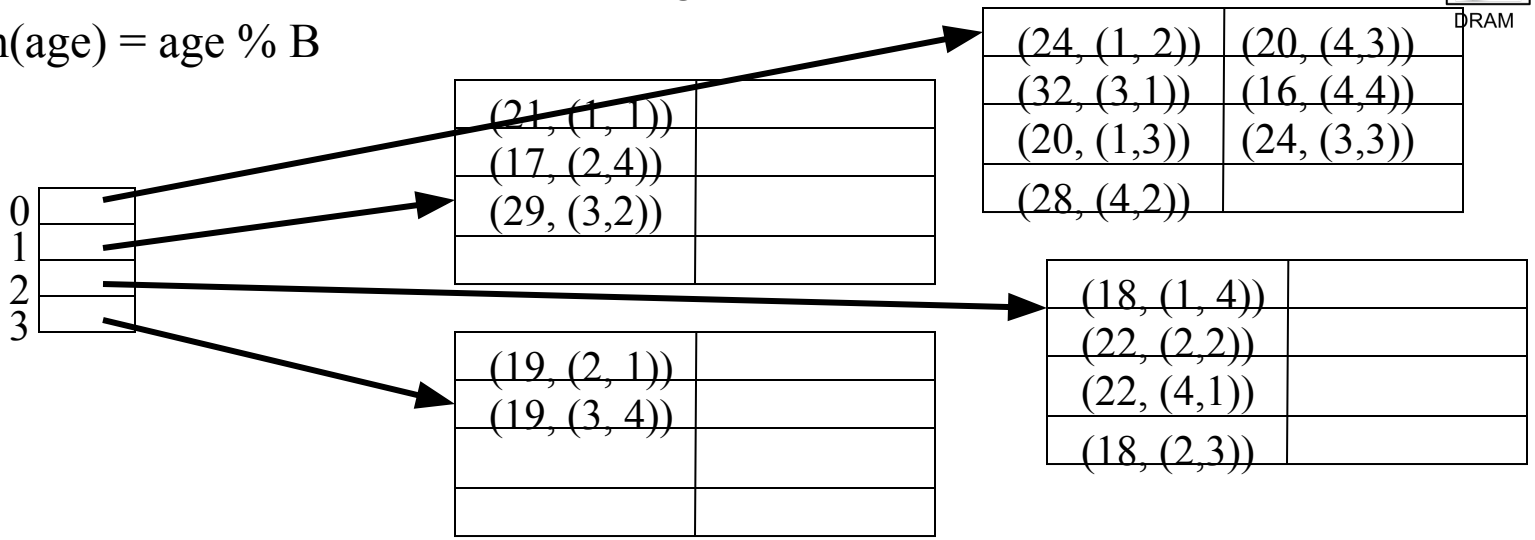
$б_{age=30}(Student)$

$б_{age=30}$(Student)

Non-Clustered Hash Index



- A non-clustered hash index on the age attribute with 4 buckets,
- h(age) = age % B



Bob, 21, 3.7, CS	Kane, 19, 3.8, ME	Louis, 32, 4, LS	Chris, 22, 3.9, CS
Mary, 24, 3, ECE	Lam, 22, 2.8, ME	Martha, 29, 3.8, CS	Chad, 28, 2.3, LS
Tom, 20, 3.2, EE	Chang, 18, 2.5, CS	James, 24, 3.1, ME	Leila, 20, 3.5, LS
Kathy, 18, 3.8, LS	Vera, 17, 3.9, EE	Pat, 19, 2.8, EE	*Shideh, 16, 4, CS

What is the MOD (%) Function?

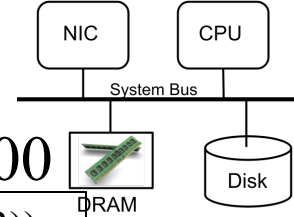
F(x) = x MOD N = x % N

- It is the remainder of x divided by N. Assume N=10:
  - F(12) = 12 % N = 2
  - F(10) = 10 % N = 0
  - F(1) = 1 % N = 1
  - F(101) = 101 % N = 1
  - F(20,511) = 20,511 % N = 1

- x MOD N maps a value x to a number between 0 and N-1.
- It is simple and perhaps the most popular hash functions out there.

Non-Clustered Hash Index

- A non-clustered hash index on the age attribute with 4 buckets,
- h(age) = age % B
- **Pointers are disk block-ids**



0	500
1	1001
2	706
3	101

500

(24, (1, 2))	(20, (4,3))
(32, (3,1))	(16, (4,4))
(20, (1,3))	(24, (3,3))
(28, (4,2))	

1001

(21, (1, 1))	
(17, (2,4))	
(29, (3,2))	

706

(18, (1, 4))	
(22, (2,2))	
(22, (4,1))	
(18, (2,3))	

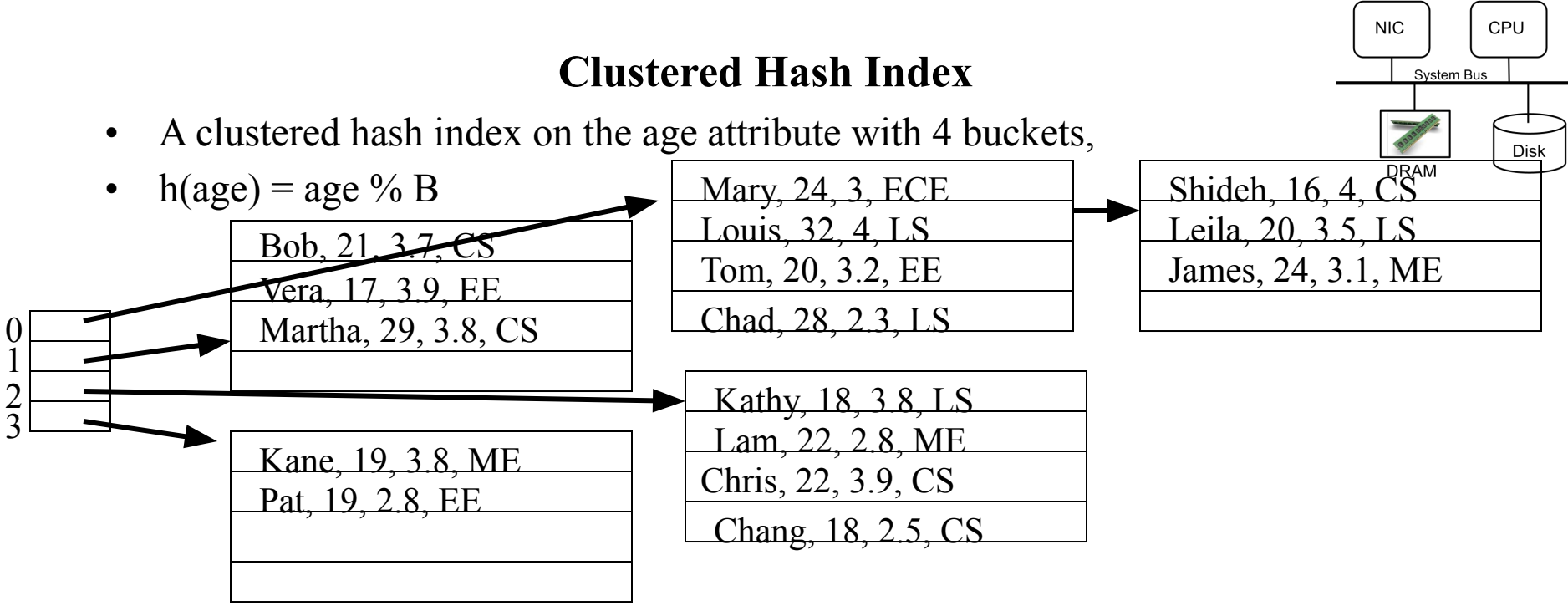
101

(19, (2, 1))	
(19, (3, 4))	

Bob, 21, 3.7, CS	Kane, 19, 3.8, ME	Louis, 32, 4, LS	Chris, 22, 3.9, CS
Mary, 24, 3, ECE	Lam, 22, 2.8, ME	Martha, 29, 3.8, CS	Chad, 28, 2.3, LS
Tom, 20, 3.2, EE	Chang, 18, 2.5, CS	James, 24, 3.1, ME	Leila, 20, 3.5, LS
Kathy, 18, 3.8, LS	Vera, 17, 3.9, EE	30 Pat, 19, 2.8, EE	Shideh, 16, 4, CS

Clustered Hash Index

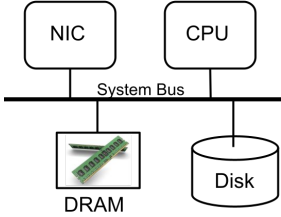
- A clustered hash index on the age attribute with 4 buckets,
- h(age) = age % B



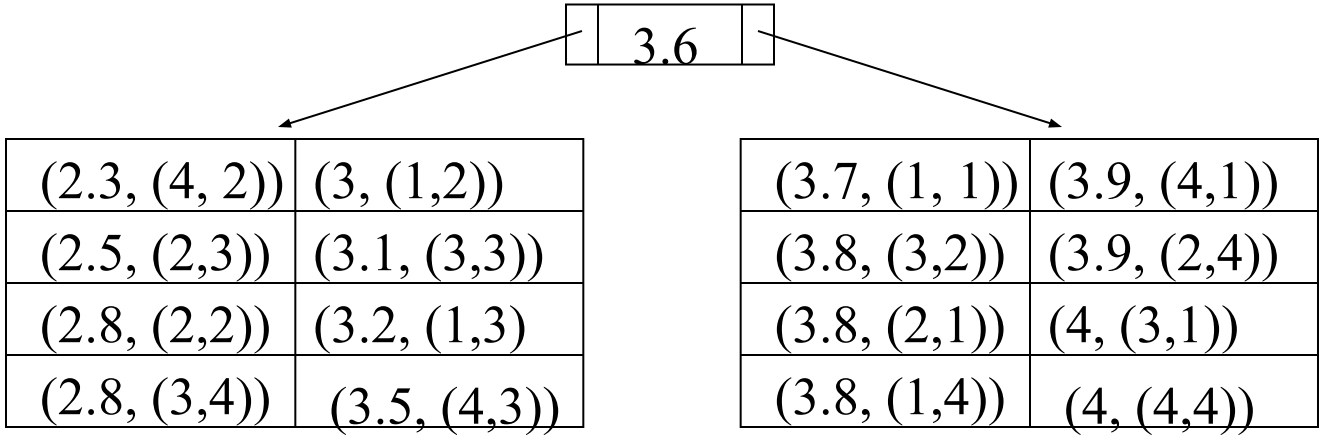
$б_{age=30}(Student)$

$б_{gpa=3.9}$(Student)

Non-Clustered Secondary B$^+$-Tree



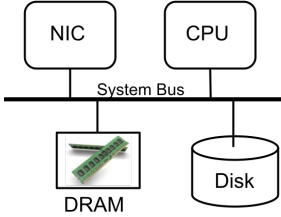
- A non-clustered secondary B+-tree on the gpa attribute



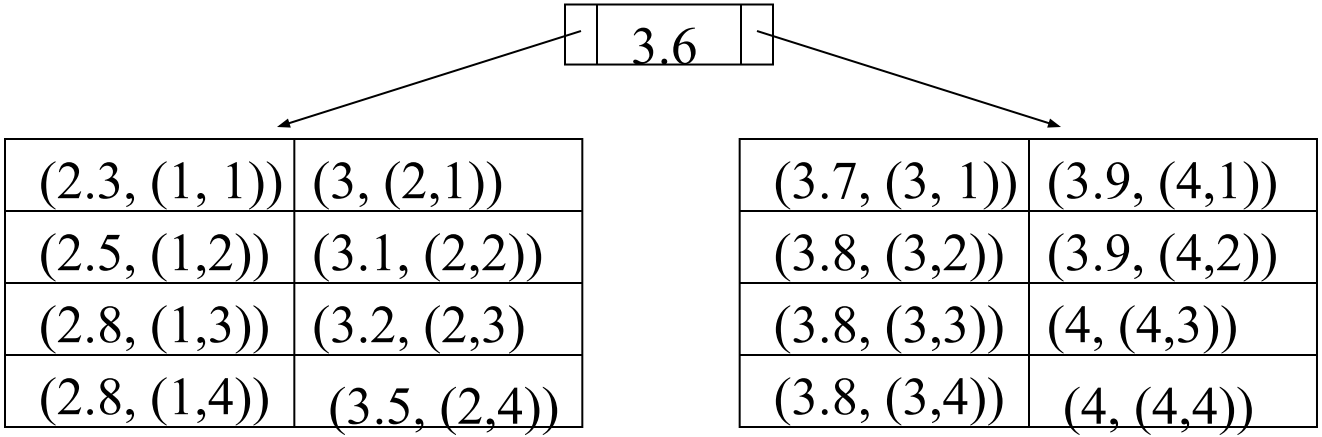
Bob, 21, 3.7, CS	Kane, 19, 3.8, ME	Louis, 32, 4, LS	Chris, 22, 3.9, CS
Mary, 24, 3, ECE	Lam, 22, 2.8, ME	Martha, 29, 3.8, CS	Chad, 28, 2.3, LS
Tom, 20, 3.2, EE	Chang, 18, 2.5, CS	James, 24, 3.1, ME	Leila, 20, 3.5, LS
Kathy, 18, 3.8, LS	Vera, 17, 3.9, EE	Pat, 19, 2.8, EE	*Shideh, 16, 4, CS

$б_{gpa=3.9}$(Student)

Non-Clustered Primary B$^+$-Tree



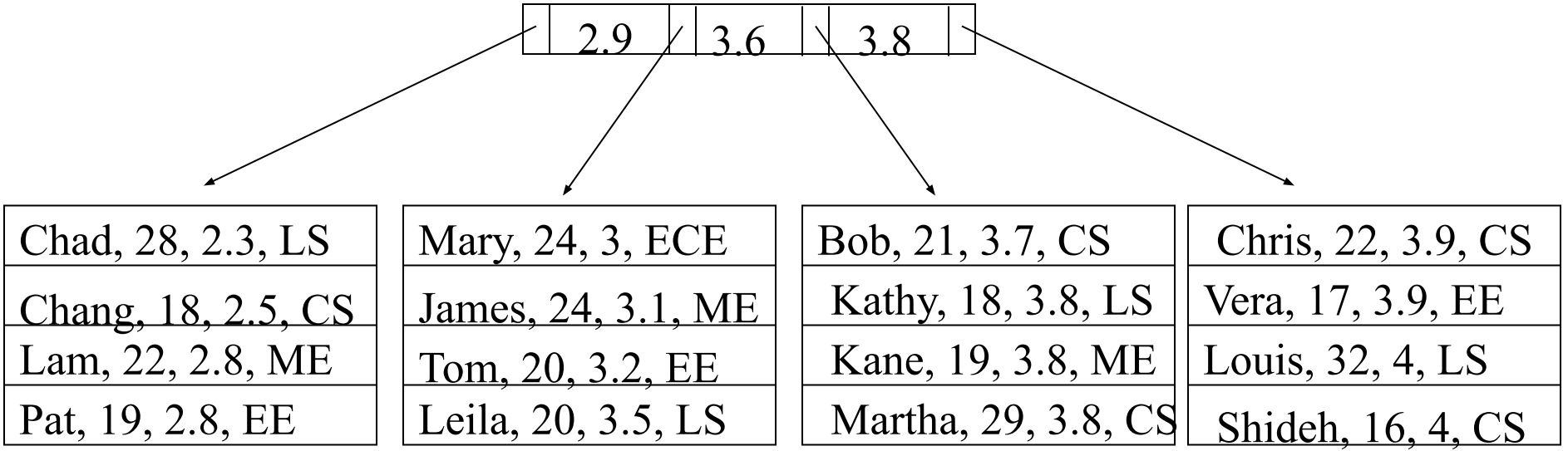
- A non-clustered primary B+-tree on the gpa attribute



Chad, 28, 2.3, LS	Mary, 24, 3, ECE	Bob, 21, 3.7, CS	Chris, 22, 3.9, CS
Chang, 18, 2.5, CS	James, 24, 3.1, ME	Kathy, 18, 3.8, LS	Vera, 17, 3.9, EE
Lam, 22, 2.8, ME	Tom, 20, 3.2, EE	Kane, 19, 3.8, ME	Louis, 32, 4, LS
Pat, 19, 2.8, EE	Leila, 20, 3.5, LS	Martha, 29, 3.8, CS	*Shideh, 16, 4, CS

Clustered B^+-Tree

- A clustered B+-tree on the gpa attribute



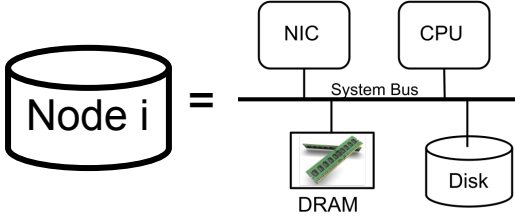
Index Structures: Parallelism May NOT Be Beneficial

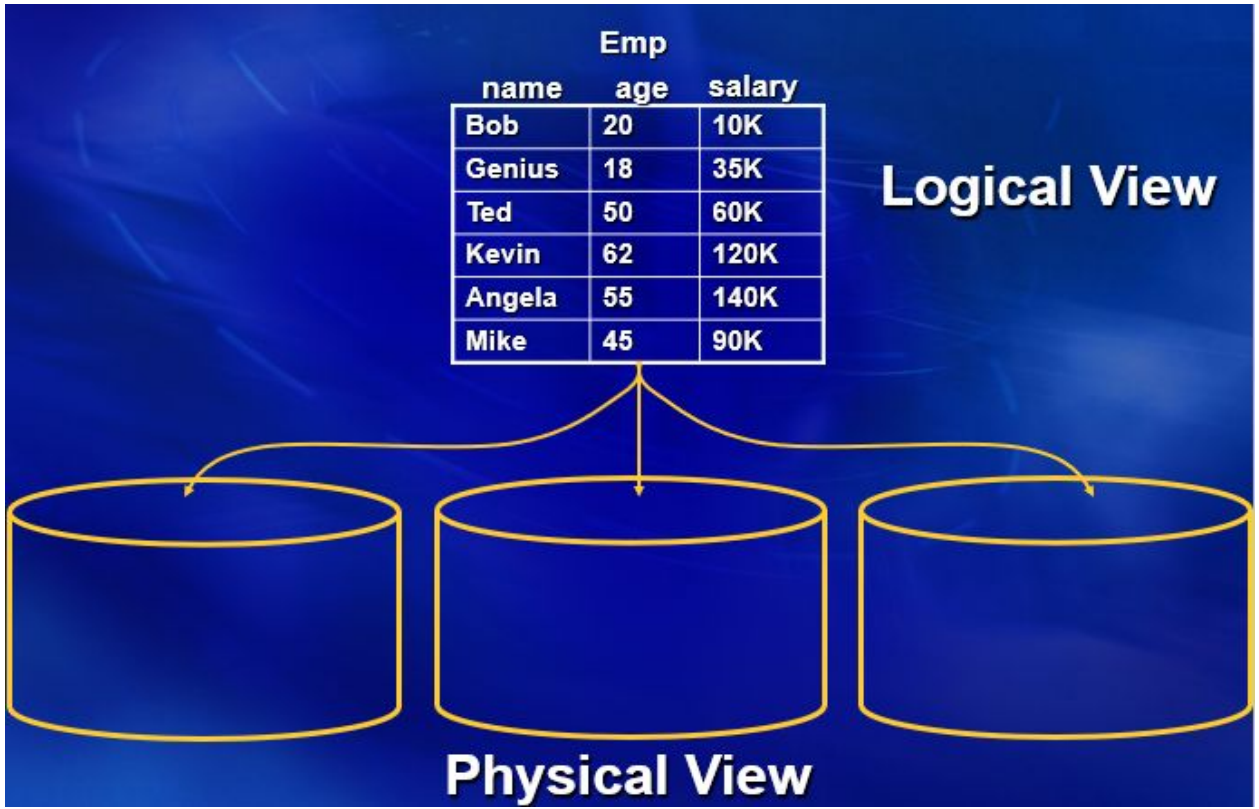
A hash index processes exact match predicates referencing the indexed attribute efficiently. A query that processes one record on one disk page should be directed to one node. Directing the query to N nodes causes N-1 nodes to search their index to find no qualifying record.

A B+-Tree index structures processes range predicates referencing the indexed attribute efficiently. A query that processes a few records on one disk page should be directed to one node.

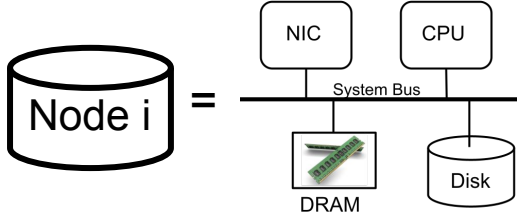
Using many nodes to process a query that is fetching and processing a few disk pages may not be appropriate. Overhead of parallelism will dominate query execution.

Horizontal Declustering/Partitioning/Sharding





Horizontal Declustering: How?



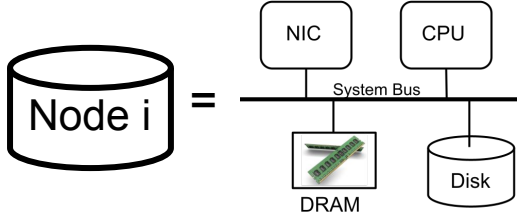
No partitioning attribute: Random and Round-robin.

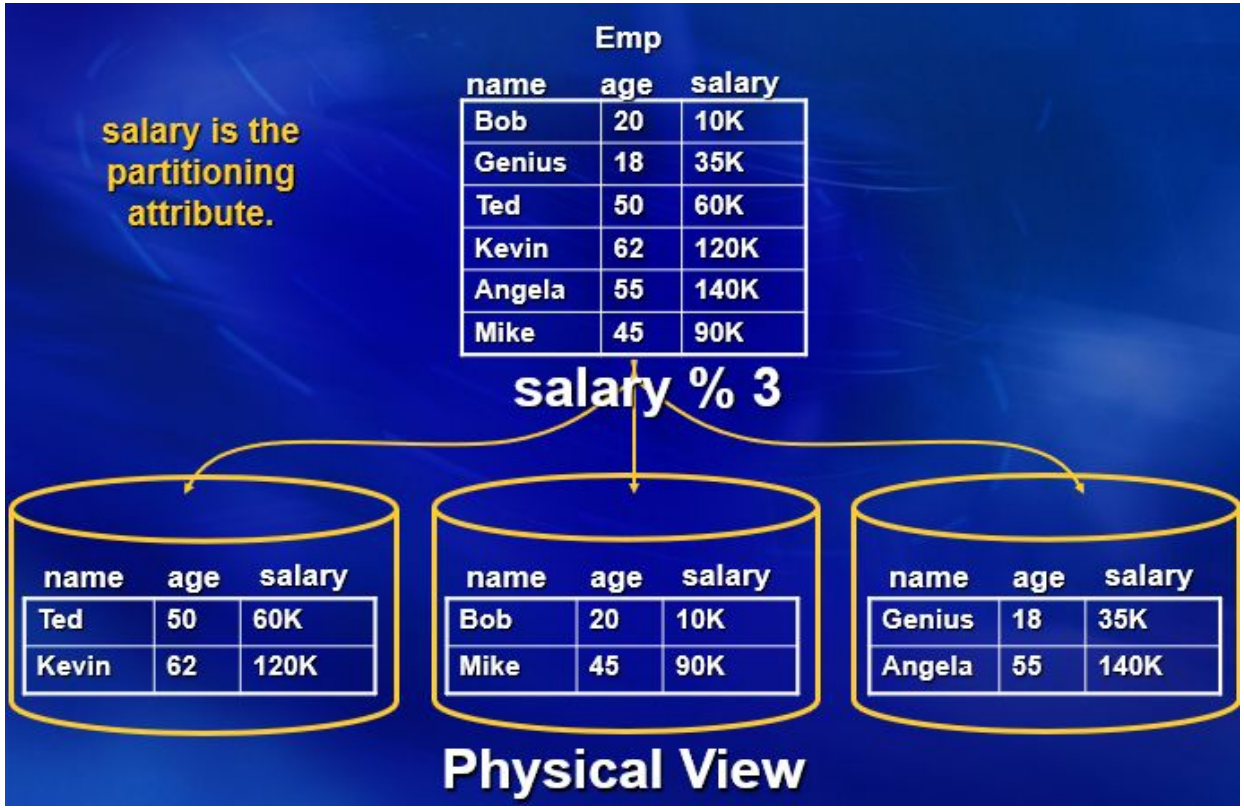
Single attribute declustering strategy:

1. Hash
2. Range

Note: The administrator must select an attribute as the partitioning attribute.

Hash Declustering/Partitioning/Sharding





Hash Declustering

Selective exact match selection predicates referencing the partitioning attribute are directed to a single node:

- Retrieve Employees with a 60K salary

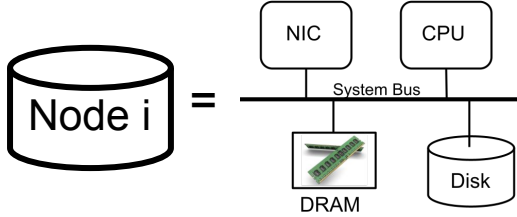
```
SELECT *
FROM   Emp
WHERE salary=60K
```

Selective range predicates referencing the partitioning attribute are directed to all nodes:

- Retrieve Employees whose salary is between 61K and 61.1K
- Query fetches a few rows using a B+-tree index AND is directed to all nodes.

```
SELECT *
FROM   Emp
WHERE sal > 61K and sal < 61.1K
```

Range Declustering/Partitioning/Sharding



salary is the partitioning attribute.

Emp

name	age	salary
Bob	20	10K
Genius	18	35K
Ted	50	60K
Kevin	62	120K
Angela	55	140K
Mike	45	90K

0-50K

name	age	salary
Bob	20	10K
Genius	18	35K

51K-100K

name	age	salary
Ted	50	60K
Mike	45	90K

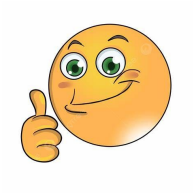
101K-∞

name	age	salary
Kevin	62	120K
Angela	55	140K

Physical View

Range Declustering

Selective exact match and range predicates referencing the partitioning attribute are directed to a subset of nodes.



```
SELECT*
FROM    Emp
WHERE sal > 61K and sal < 61.1K
```

What about range predicates referencing the partitioning attribute that are **not** selective and should be processed using several nodes. Example query fetches many disk pages from the B+-tree index on one node.

```
SELECT*
FROM    Emp
WHERE age<50K
```

Hash Declustering

What about non selective exact match selection predicates? Those referencing the partitioning attribute are still directed to a single node, e.g.,

- What if there are many duplicates 60K?

```
SELECT *
FROM   Emp
WHERE salary=60K
```

Selective Range predicates referencing the partitioning attribute are directed to all nodes:

- Retrieve Employees whose salary is between 61K and 61.1K
- Query fetches a few rows using a B+-tree index AND is directed to all nodes.

```
SELECT *
FROM   Emp
WHERE sal > 61K and sal < 61.1K
```

Hash Declustering

Why is it bad to send a query that fetches many disk pages (i.e., does a lot of work) to 1 node instead of all N nodes?

Hash Declustering

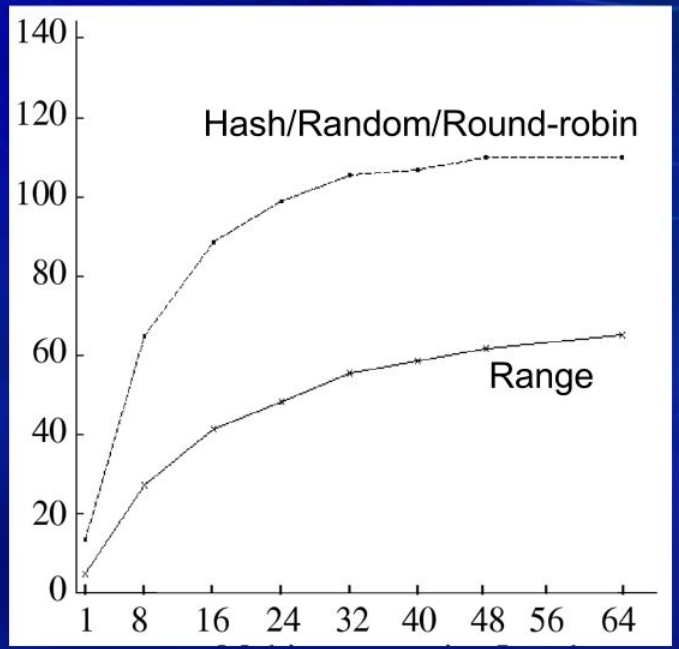
Why is it bad to send a query that fetches many disk pages (i.e., does a lot of work) using 1 node instead of all N nodes?

1. Increases service time of the query.
   a. Service Time μ (mu) = time to process a query with no other load on the system.
   b. Response Time τ (tau) = Service Time + Queuing Delay attributed to other concurrently executing queries/transactions.
2. Results in formation of hotspots and bottlenecks that reduce the system throughput.
   a. Throughput λ (lambda): The rate at which a system processes requests when it is fully utilized. It is independent of the queuing delay. In a closed system, throughput is the reciprocal of the service time, $\lambda = 1/\mu$


1
2
3
6 Ideal cases
R1
R2
R3
R1
R3
R2
R2
R1
R3
R2
R3
R1
R3
R2
R1
{R1, R3}
R2
{R1, R3}
R2
{R1, R3}
R2
R2
{R1, R3}
R2
{R1, R3}
R2
{R1, R3}
R2
{R2, R3}
R1
{R2, R3}
R1
{R2, R3}
R1
R1
{R2, R3}
R1
{R2, R3}
R1
{R2, R3}
{R2, R1}
R3
{R2, R1}
R3
{R2, R1}
R3
R3
{R2, R1}
R3
{R2, R1}
R3
{R2, R1}
{R1, R2, R3}
{R1, R2, R3}
{R1, R2, R3}
27 ways to assign 3 requests to the 3 nodes! Only 6 result in a uniform distribution of requests.

Range Declustering

- Range selection predicate using a clustered B+-tree, 1% selectivity (1000 records)



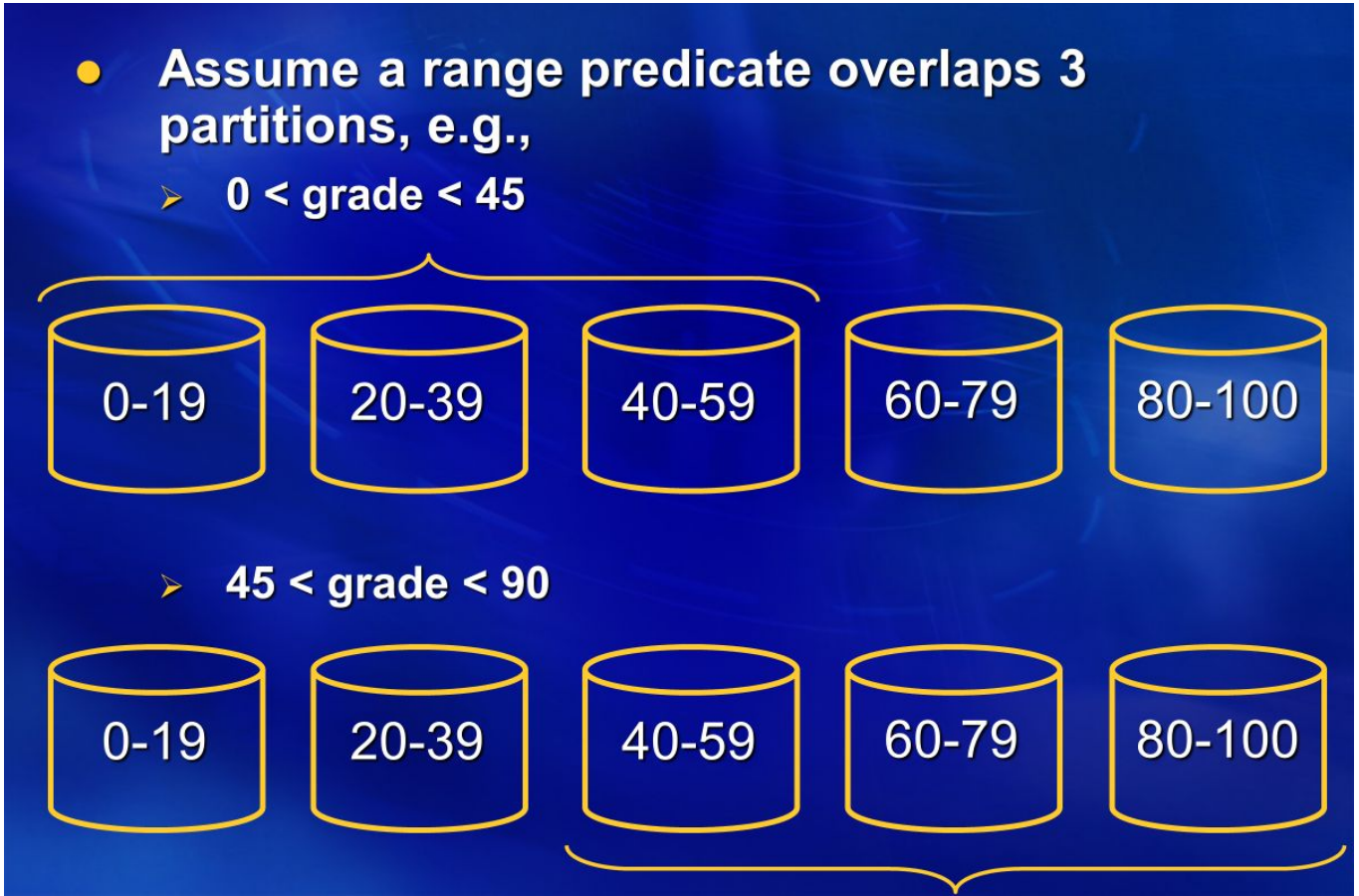
Why Range Performs Poorly?

- Note: Range performed poorly because the query (1% selection) imposed a high workload onto a node!
  - For a query with minimal (0.01% selection) workload requirement, Range is ideal!
- Two reasons:
  - Random generation of selection predicates does NOT mean uniform distribution of workload across nodes.
  - The number of ranges is the same as the number of nodes causing the tail-end servers to observe a lower load.

Range has a tail end effect that results in load imbalance

- **Simple range partitioning may lead to load imbalance for queries with high selectivity:**
  - **Low performance: increased response time and low system throughput.**
- **Consider a table that maintains the grade of students for different exams, range partitioned on the grade.**

Query may not wrap around from 80-100 to 0-19



Load Imbalance

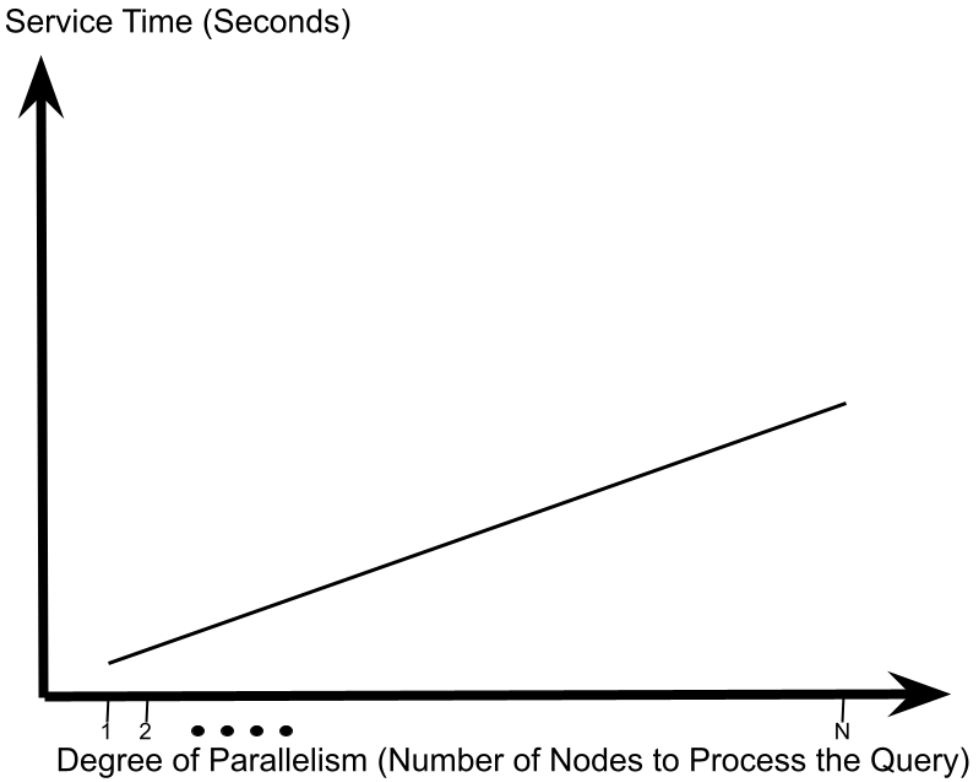
- **Higher response time because 2 nodes sit idle while 3 nodes process the query (assuming overhead of parallelism is negligible).**

Load Imbalance: Node 3 Becomes the Bottleneck

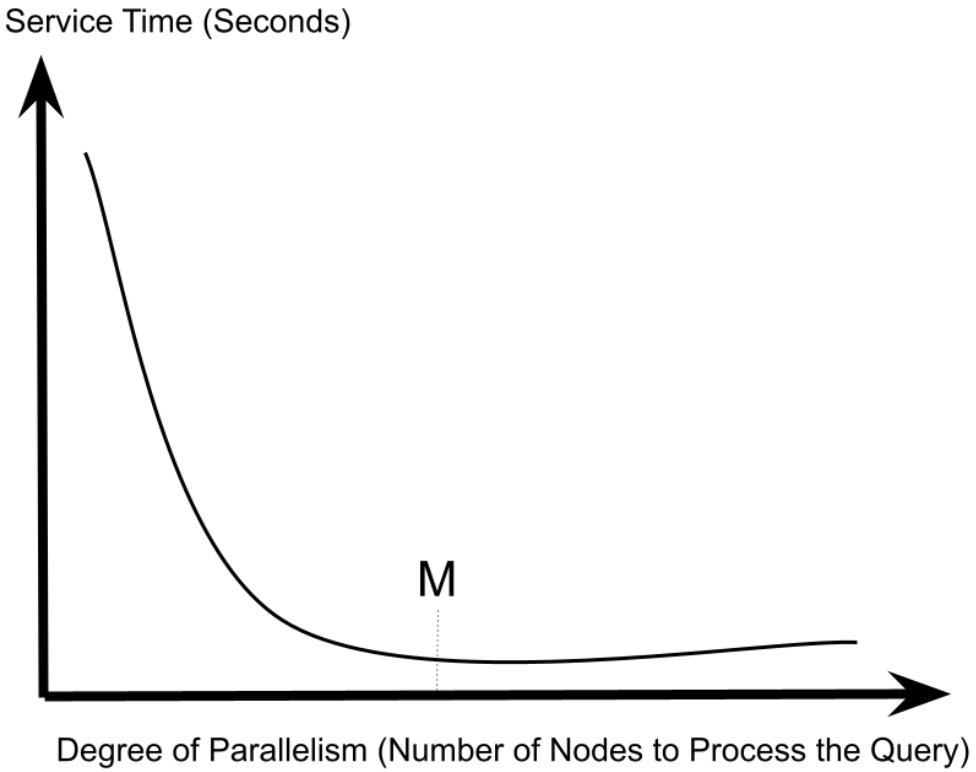
- **Lower throughput because node 3 becomes a bottleneck.**
  - Assuming even distribution of access to ranges, when node 3 is utilized 100%, nodes 2 and 4 have a 66% utilization, while nodes 1 and 5 are utilized 33%.

HRPS: Hybrid Range Partitioning Strategy

Observation 1: Service Time of a Query that Processes a Few Disk Pages



Observation 2: Service Time of a Query that Processes Many Disk Pages



M = Desired Degree of Parallelism for a Query

How? Compute a function that describes service time as a function of M.

$$RT(M) = \frac{CPU_{Ave} + Disk_{Ave} + Net_{Ave}}{M} + M * CP$$

M is the first derivative of the function.

$$M = \sqrt{\frac{CPU_{Ave} + Disk_{Ave} + Net_{Ave}}{CP}}$$

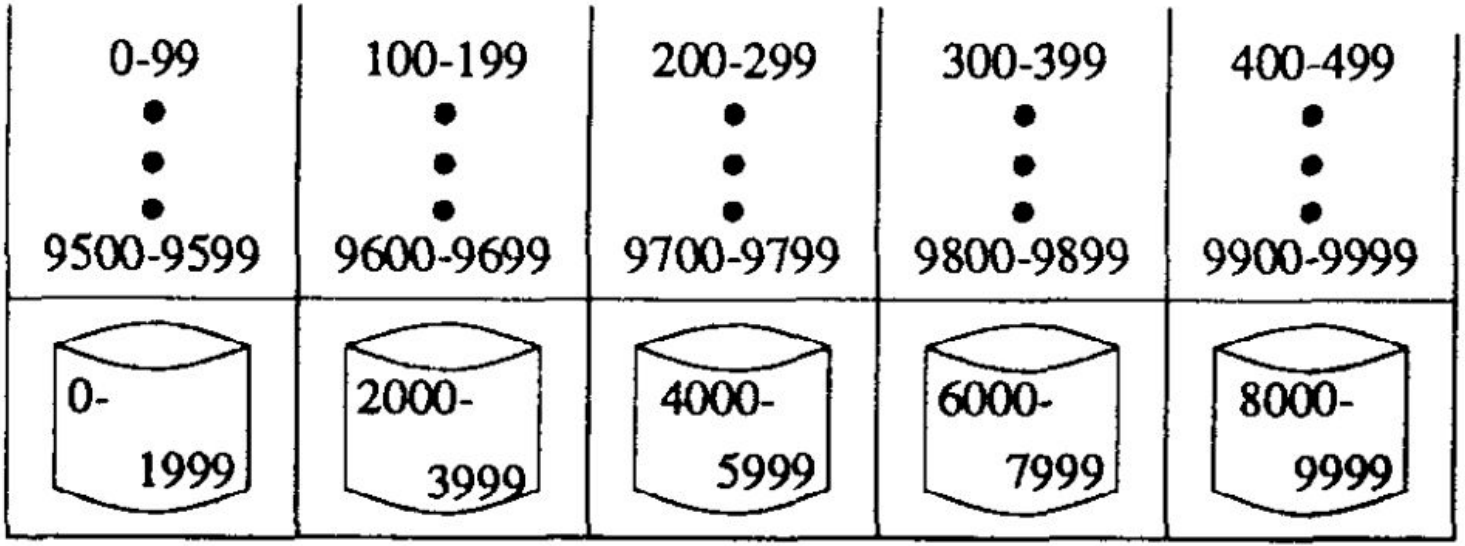
Example

tegies, consider the following example. Assume a 10,000 tuple relation with unique values for the partitioning attribute ranging from 0 to 9,999 and that the "average" query accessing this relation uses a range predicate on the partitioning attribute to retrieve and process 500 tuples ($TuplesPerQ = 500$). Also, assume that the optimal performance is achieved when 5 processors are used ($M = 5$). Thus, the cardinality of each fragment is 100 tuples ($FC = \frac{TuplesPerQ}{M} = 100$) and the relation will be partitioned into 100 fragments. Finally, assume that the alternative partition-

Example, M = N

hybrid-range (HRPS)

range



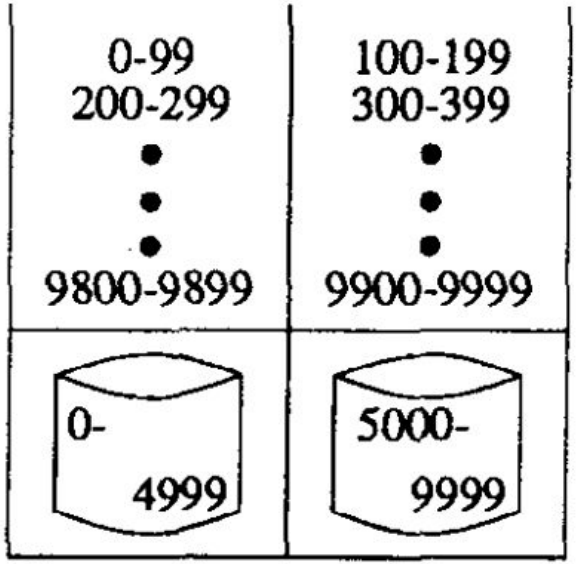
a. M = N

- How is our example query processed with range?
- How is our example query processed with HRPS?

Example, M > N

hybrid-range (HRPS)

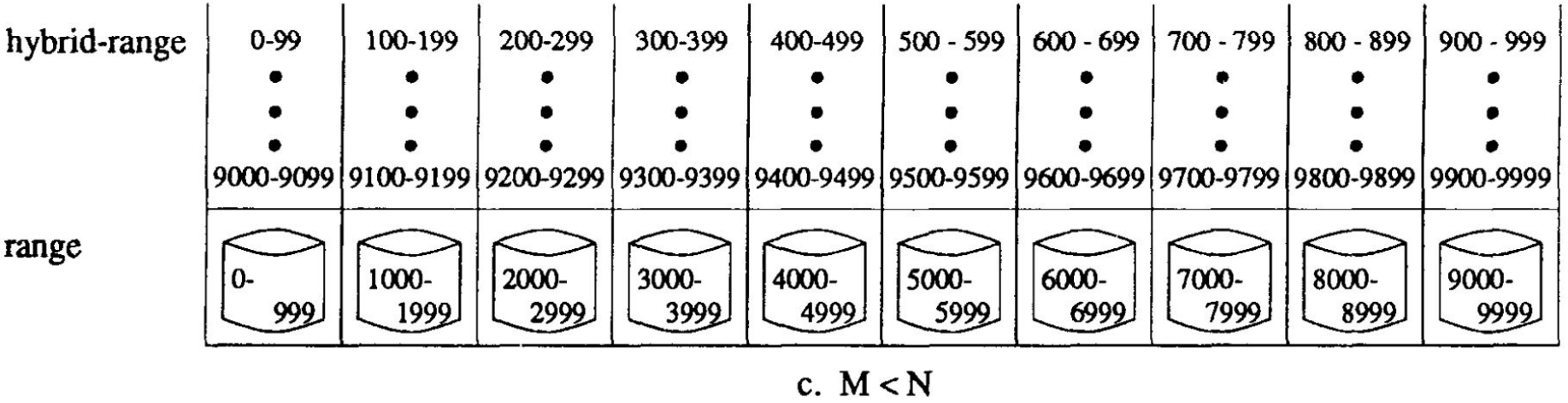
range



b. M > N

- How is our example query processed with range?
- How is our example query processed with HRPS?

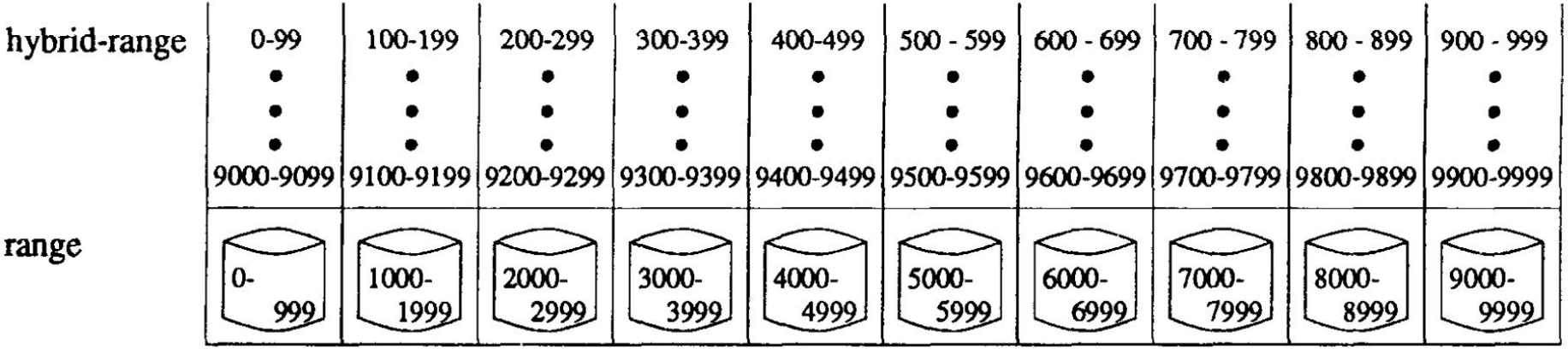
Example, M < N



c. M < N

- How is our example query processed with range?
- How is our example query processed with HRPS?
- Which will provide a superior response time?
- Which will provide a higher throughput?

Overhead of Searching a Directory?



c. M < N

Linear search:

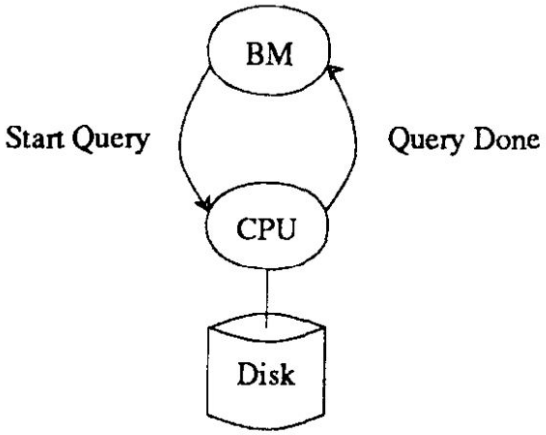
$$RT(M) = \frac{CPU + Disk + Net}{M} + M * CP + \frac{M * Cardinality(Rel) * CS}{TuplesPerQ}$$

$$M = \sqrt{\frac{CPU + Disk + Net}{CP + \frac{Cardinality(Rel) * CS}{TuplesPerQ}}}$$

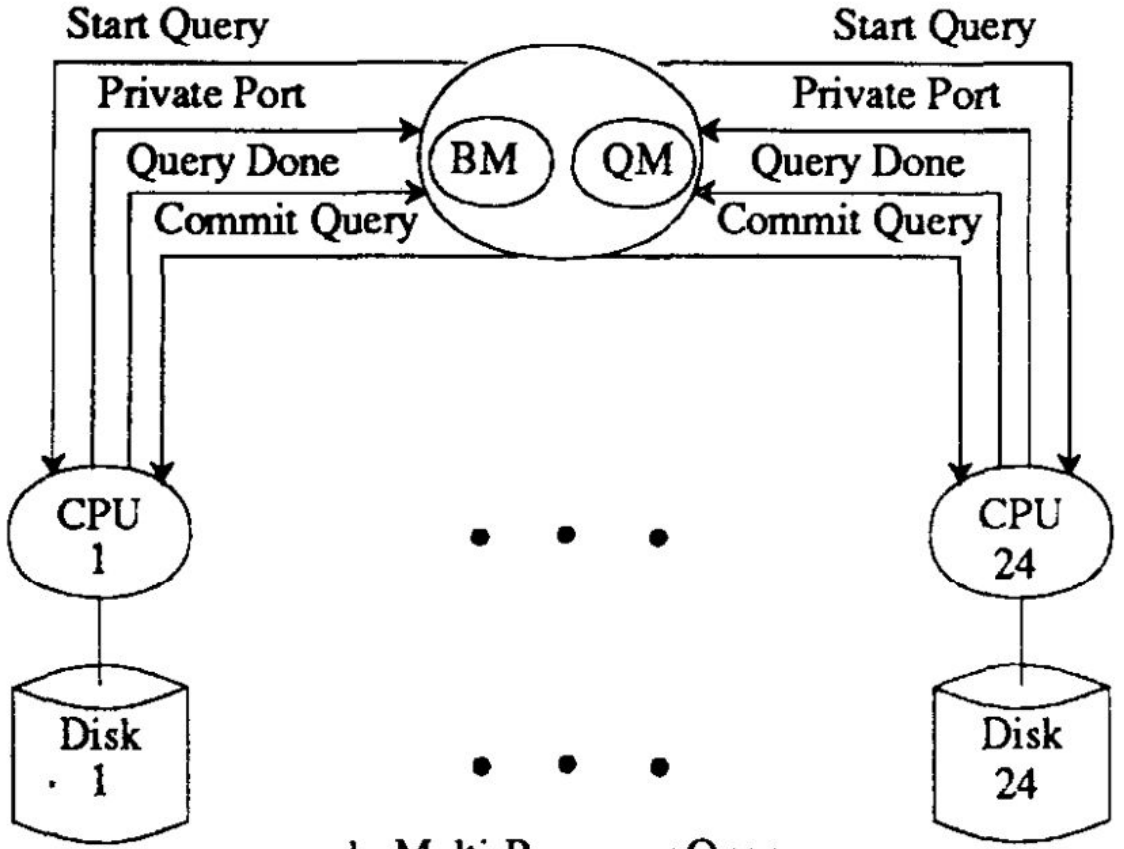
Binary Search

$$M = \frac{-\frac{CS}{\ln 2} + \sqrt{(\frac{CS}{\ln 2})^2 + (4 * CP * (CPU + Disk + Net))}}{2 * CP}$$

Query Execution



a. Single Processor Query



b. Multi-Processor Query.

HRPS

1. Let M be the desired degree of parallelism to process the average query in the workload.
2. Let TuplesPerQ be the average number of tuples processed by the average query.
3. Desired number of tuples processed by one of the M servers is FC = TuplesPerQ/M.
4. Set the cardinality of each fragment to FC.
5. Number of fragments F = Cardinality(Rel)/FC.
6. Construct F ranges, each with approximately FC records.

Fetch 10 Records Using a Clustered B+-tree Index

1. Let M be the desired degree of parallelism to process the average query in the workload. **M=0.057.**
2. Let TuplesPerQ be the average number of tuples processed by the average query. **TuplesPerQ = 10.**
3. Desired number of tuples processed by one of the M servers is FC = TuplesPerQ/M. **FC = 10/0.057 = 175.**
4. Set the cardinality of each fragment to FC.
5. Number of fragments F = Cardinality(Rel)/FC. **1 Million/175 = 5,700**
6. Construct F ranges, each with approximately FC records.

Range Query: Fetch 10 Records Using a Clustered B+-tree Index

This is our example query with 5700 fragments.

Throughput increases linearly from multiprogramming level of 1 to 12. It becomes sub-linear after 13. Why?

Why is the throughput with Hash so low?

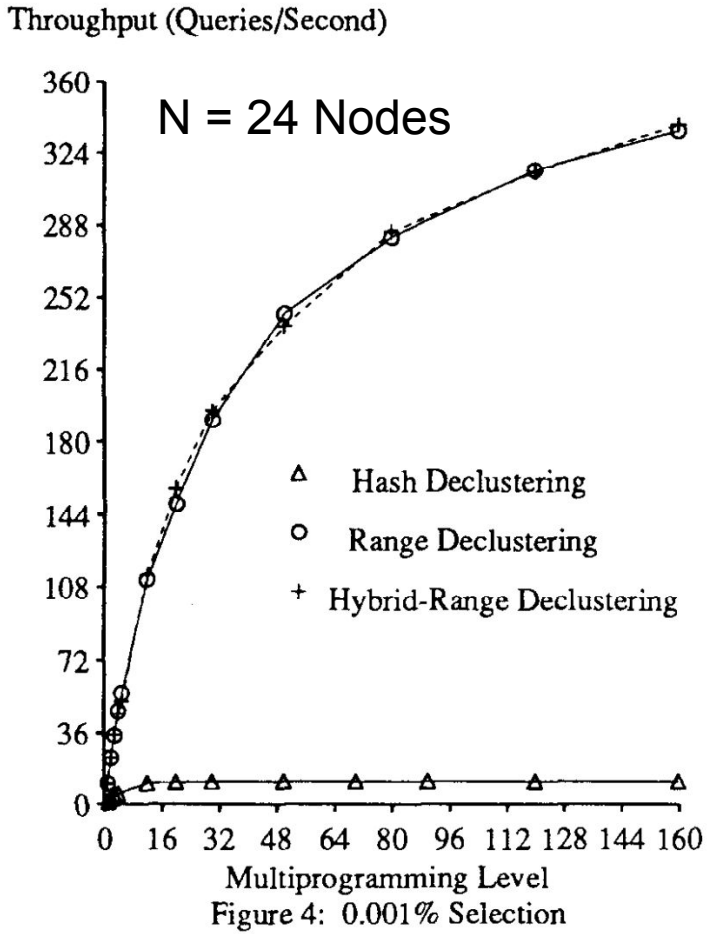


Figure 4: 0.001% Selection

Range Query: Fetch 10K Records Using a Clustered B+-tree Index

M=90. HRPS constructs 895 fragments.

Range constructs 24 fragments. The query overlaps ranges assigned to 3 nodes and uses 3 nodes at a time.

How many nodes does hash use to process this query?

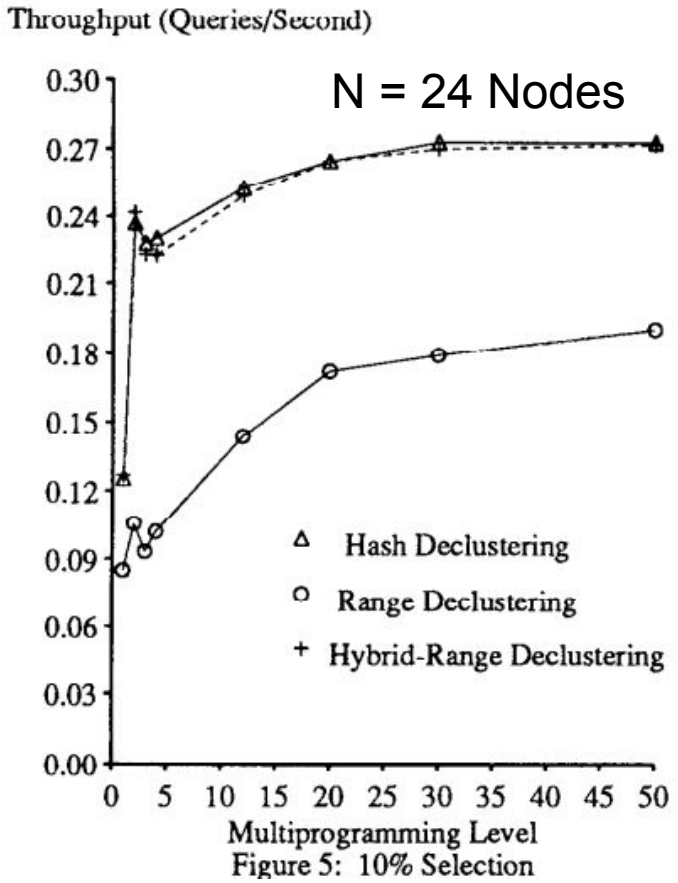


Figure 5: 10% Selection

Range Query: Fetch 10K Records Using a Clustered B+-tree Index

M=90. HRPS constructs 895 fragments.

With the range partitioning strategy, the throughput increases only slightly from a multiprogramming level of one to two because every time two queries are directed to the same processor the response time of each query increases significantly as the SCSI cache becomes ineffective. Since a large number of disk pages are processed by a query at a single processor, when the SCSI cache becomes ineffective, the performance of the system degrades significantly. The throughput of the system increases at higher multiprogramming levels as previously idle resources become more fully utilized.

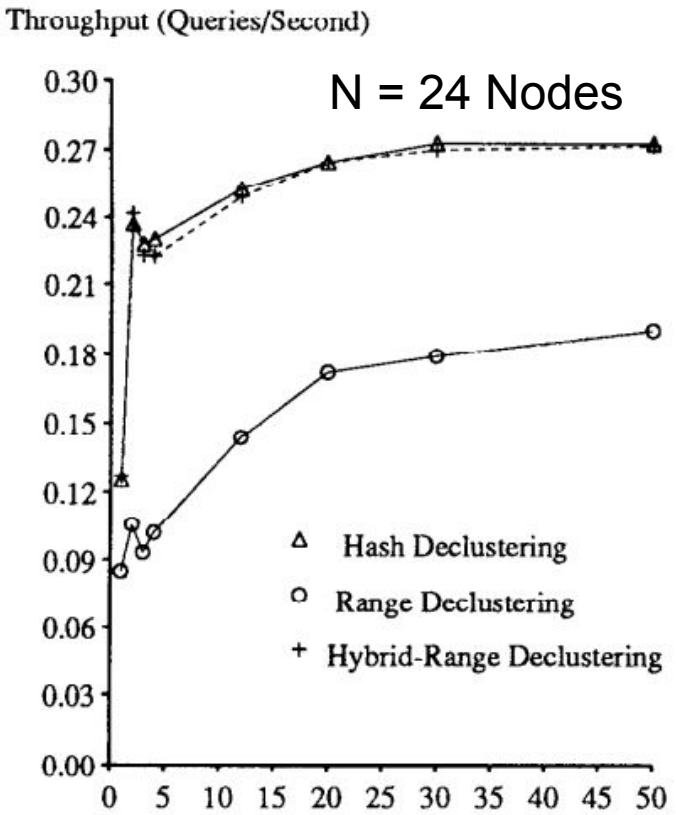


Figure 5: 10% Selection

Range Query: Fetch 10K Records Using a Clustered B+-tree Index

M=90. HRPS constructs 895 fragments.

With the range partitioning strategy, the throughput increases only slightly from a multiprogramming level of one to two because every time two queries are directed to the same processor the response time of each query increases significantly as the SCSI cache becomes ineffective. Since a large number of disk pages are processed by a query at a single processor, when the SCSI cache becomes ineffective, the performance of the system degrades significantly. The throughput of the system increases at higher multiprogramming levels as previously idle resources become more fully utilized.

Why do not we see the impact of the disk cache with the range predicate that fetched 10 records?

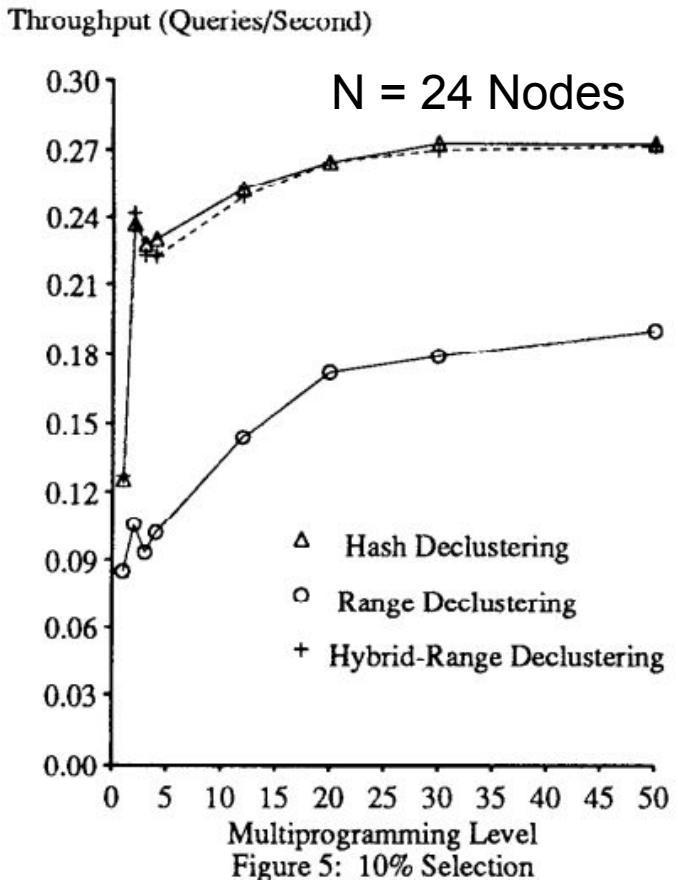


Figure 5: 10% Selection

HRPS: Other Features

Support for small relations: HRPS constructs fewer fragments than nodes, declustering small tables across a subset of the nodes.

Support for tables with many duplicates for the partitioning attribute value: HRPS will construct multiple duplicate ranges, assigning them to different nodes. Can range and hash partitioning strategy do the same?

Is There Room For Improvement?

Yes, what happens when:

- The underlying hardware platform is heterogeneous? There will be multiple functions. Will the 1st derivative of one function work for all queries?
- What happens with queries that reference different attributes of a table? Need for multiple attributes to partition a table.

MAGIC

by Shahram Ghandeharizadeh, David DeWitt, and Waheed Qureshi
in SIGMOD 1992, see https://doi.org/10.1145/141484.130293 for details.

CSCI 585

Multi-Attribute Declustering (E.g.)

- Recall the Emp(name, age, salary) table.
- Workload consists of two queries, each with a 50% frequency of occurrence:
  - Query A, range query referencing the age attribute. On average, retrieves 5 tuples.
    - Retrieve Emp where age > 21 and age < 22.
  - Query B, range query referencing the salary attribute. On average, retrieves 10 tuples.
    - Retrieve Emp where salary > 50K and salary < 50.5K
  - Access methods:
    - A non-clustered B^+-tree index on age
    - A clustered B^+-tree index on salary
- Ideally, both queries should be directed to one node.

Multi-Attribute Declustering (E.g. Cont...)

- Range decluster Emp using age as the partitioning attribute.
- Assuming a system configured with nine nodes, the number of employed nodes is:

	Range	Ideal
A	50% * 1	50% * 1
B	50% * 9	50% * 1
Average	5.5	1

MAGIC

- Construct a multi-attribute grid directory on the Emp table

Age \ Salary	0-20	21-25	26-30	31-35	36-40	41-70
10-20	1	1	4	4	7	7
21-25	1	1	4	4	7	7
26-30	2	2	5	5	8	8
31-35	2	2	5	5	8	8
36-40	3	3	6	6	0	0
41-60	3	3	6	6	0	0

- Each dimension corresponds to a partitioning attribute.
- Each cell represents a fragment of the relation.

MAGIC (Low Correlation)

- Low correlation between salary and age attribute values:

1	1	4	4	7	7
1	1	4	4	7	7
2	2	5	5	8	8
2	2	5	5	8	8
3	3	6	6	0	0
3	3	6	6	0	0

	MAGIC	Range	Ideal
A	50% * 3	50% * 1	50% * 1
B	50% * 3	50% * 9	50% * 1
Avg	3	5.5	1

MAGIC (High Correlation)

- High correlation between salary and age attribute values:

1	1	4	4	7	7
1	1	4	4	7	7
2	2	5	5	8	8
2	2	5	5	8	8
3	3	6	6	0	0
3	3	6	6	0	0

	MAGIC	Range	Ideal
A	50% * 1	50% * 1	50% * 1
B	50% * 1	50% * 9	50% * 1
Avg	1	5.5	1

BERD

- Range partition Emp using the salary attribute.
- For the age attribute, construct an auxiliary relation containing:
  1. The age attribute value of each record
  2. Node containing that record
- Range partition the auxiliary relation using the age attribute value.

BERD

salary is the primary partitioning attribute.

Emp

name	age	salary
Bob	20	10K
Genius	18	35K
Ted	50	60K
Kevin	62	120K
Angela	55	140K
Mike	45	90K

Physical View

BERD, Auxiliary relation

Auxiliary relation

age	Node
20	0
18	0
50	1
45	1
62	2
55	2

0-50K

name	age	salary
Bob	20	10K
Genius	18	35K

51K-100K

name	age	salary
Ted	50	60K
Mike	45	90K

101K-∞

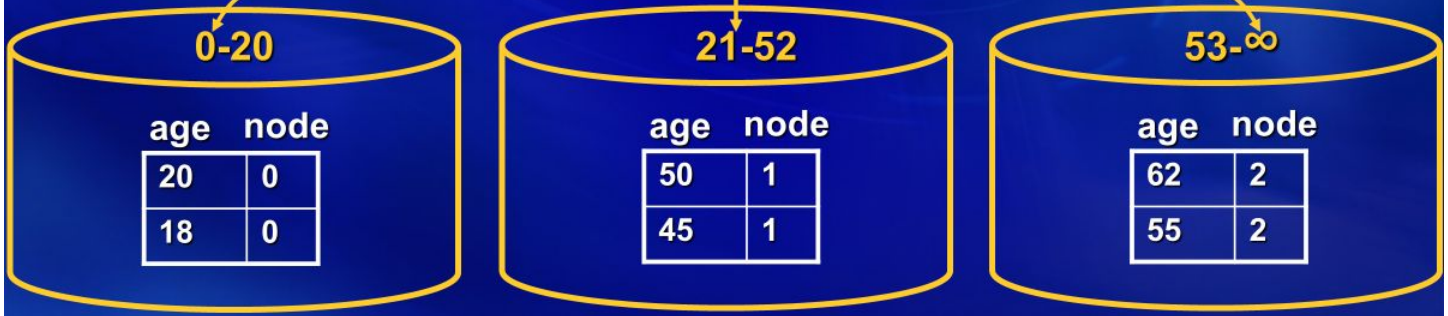
name	age	salary
Kevin	62	120K
Angela	55	140K

BERD, Auxiliary relation

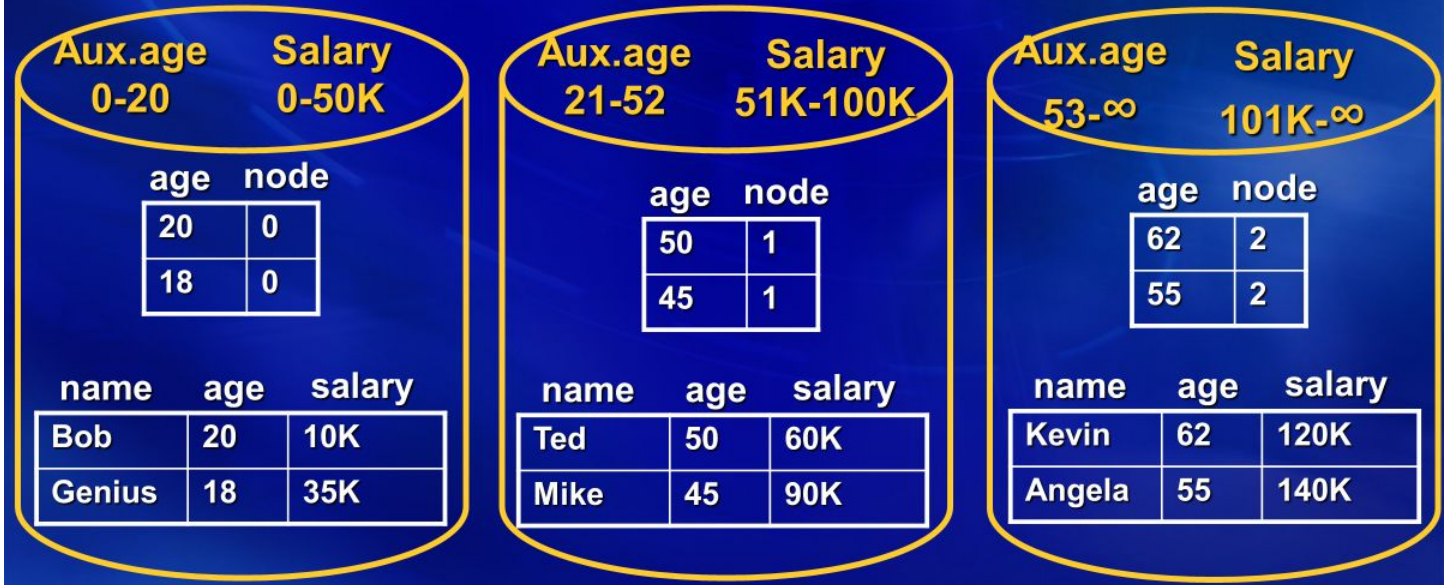
Range partition auxiliary relation using the age attribute.

Auxiliary relation

age	Node
20	0
18	0
50	1
45	1
62	2
55	2



BERD, Auxiliary relation



BERD (Cont…)

- High correlation between age and salary attribute values:

	BERD	Range	Ideal
A	50% * 1	50% * 1	50% * 1
B	50% * 1	50% * 9	50% * 1
Avg	1	5.5	1

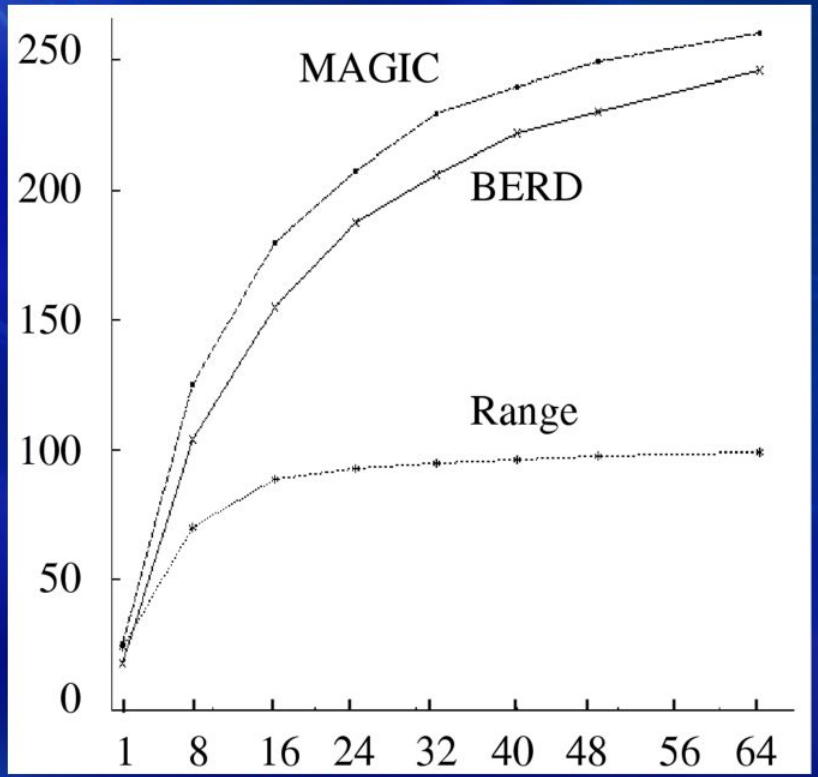
BERD (Cont…)

- Low correlation between age and salary attribute values:

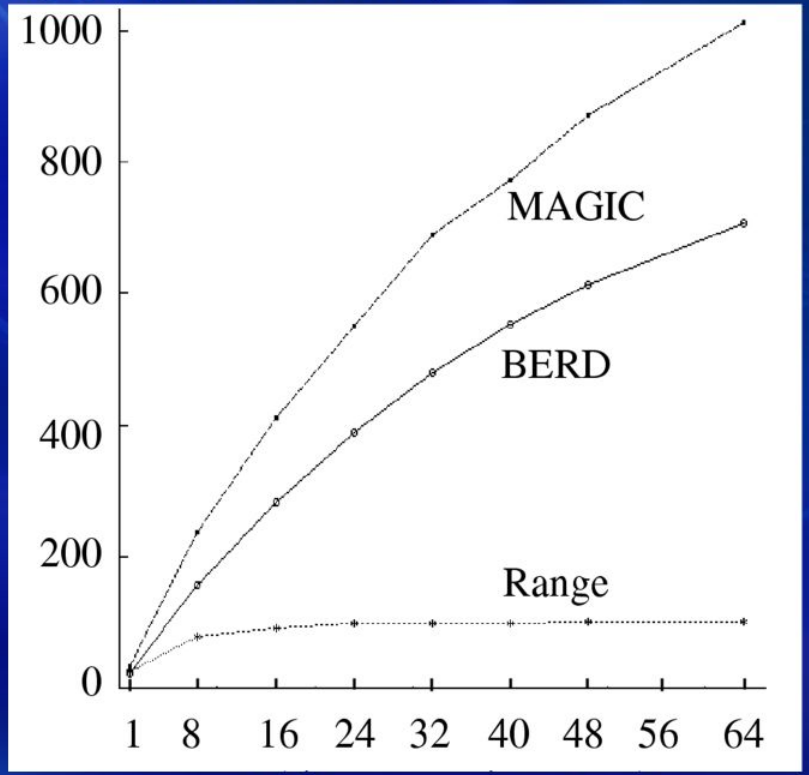
	BERD	Range	Ideal
A	50% * 1	50% * 1	50% * 1
B	50% * 9	50% * 9	50% * 1
Avg	5.5	5.5	1

Is it possible to avoid lookup in the auxiliary table?

Low-Low Query Mix (Low Correlation)



Low-Low Query Mix (High Correlation)



Advantages of MAGIC

- Provides a superior performance when compared to BERD and Range
- Constructs the grid directory using the workload of the relation. Changes the shape of the grid directory in order to compensate for the different frequencies of access to the partitioning attributes.
- Minimizes the overhead of parallelism.
- Supports partial declustering of a relation in large systems.

Performance Modeling

When to Issue Requests?

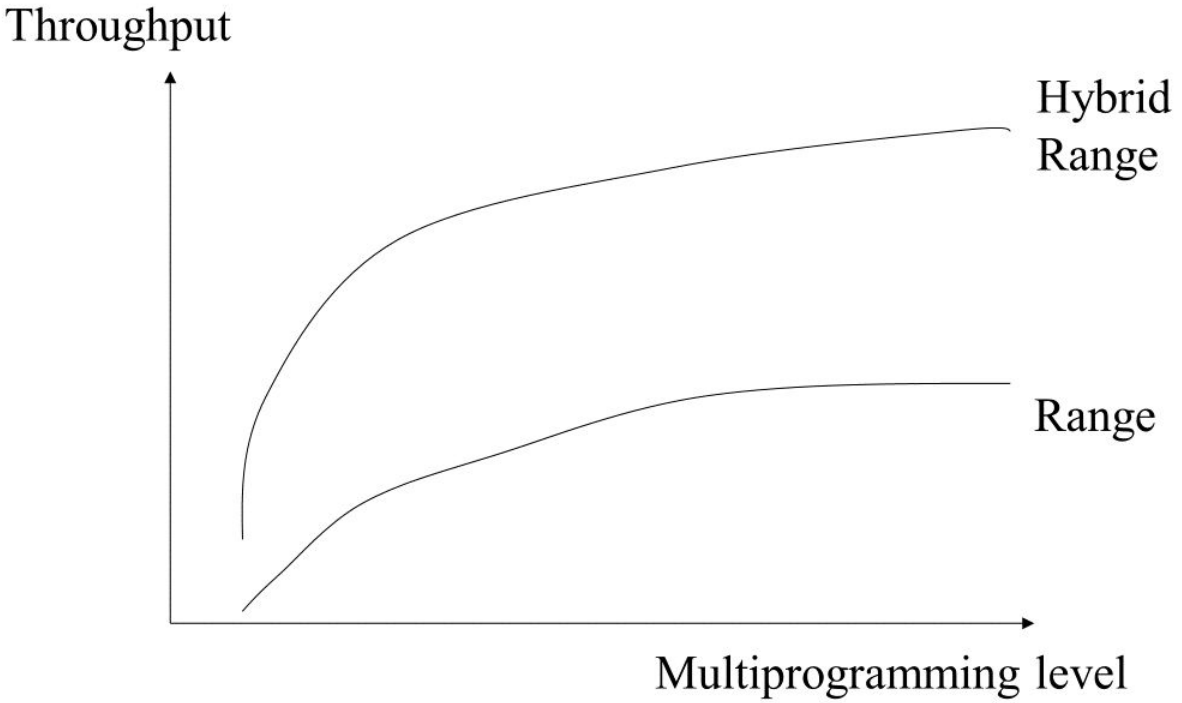
- It depends on the assumed simulation model:
  - Closed simulation models:

    - Consists of a fixed number of clients.
    - A client issues a request and does not issue another until its pending request is serviced.
    - The client may wait a pre-specified amount of time before issuing a new request. This delay is termed "think time".
  - Open simulation models:

    - A process issues an average number of requests per unit of time. This is termed request arrival rate. It is denoted as lambda: λ

Closed Simulation Models

A technique is better if it provides a higher throughput
relative to the other technique as a function of the number of clients.



Open Simulation Models

A technique is better if it supports a higher arrival rate without formation of queues. A slower technique results in queues with lower arrival rates.